

ESWIN

BSP 快速入门指南

Rev 2.0

2026-3-26

声明

知识产权声明

除非另有说明，ESWIN 保留对本产品的所有知识产权。未经 ESWIN 授权，不得对本产品进行反向工程、反编译，不得修改、改编、制作、分发本产品的衍生作品。

ESWIN 保留更新本文档或改进其中所提及的产品或服务的权利。除非另有说明，本文档中的所有陈述、信息及建议均按“现状”提供，ESWIN 不对此做出任何形式的担保、保证或承诺，无论明示或默示。

“ESWIN”商标和其他 ESWIN 标识，为北京奕斯伟计算技术股份有限公司及（或）其母公司所有的商标，未经授权，不得擅自使用。

责任限制

除本文档或交易文书另有约定外，ESWIN 不提供任何明示、暗示或法律上的保证，包括但不限于适销性、适用性、所有权或非侵权的任何暗示或明示保证，或因任何建议、规格、样品、交易、惯例或贸易惯例而产生的任何保证。

ESWIN 不对任何意外性、惩罚性、间接性的损害承担责任，包括但不限于利润损失、业务中断或采购任何替代商品或服务而产生的成本。

修订记录

文档版本	日期	说明
Rev 2.0	2026/03/26	增加 bootloader 移植应用开发指南
Rev 1.0	2025/07/10	初始版本

目录

声明 1

 知识产权声明 1

 责任限制 1

修订记录 2

1. 概述 7

2. 硬件平台介绍 8

 2.1. EIC7700 硬件平台 8

 2.1.1. HiFive Premier P550 8

 2.1.2. EBC77 系列单板计算机 (SBC) 10

 2.1.3. EIC7700FG EVB LP5 12

 2.2. EIC7702 硬件平台 13

 2.2.1. EBC7702 Mini-DTX 13

 2.3. 硬件平台用户手册 15

3. 系统编译指南 16

 3.1. 编译环境说明 16

 3.2. Ubuntu22.04 原生编译环境搭建 16

 3.2.1. 安装通用编译依赖 16

 3.2.2. 安装 RISC-V 交叉编译工具链 16

 3.3. Docker 编译环境搭建 17

 3.4. Debian 系统编译 17

 3.5. Ubuntu 系统编译 18

4. Bootloader 开发指南 20

 4.1. 启动架构与 Bootloader 概述 20

 4.1.1. Linux 系统启动流程 20

 4.1.2. 二级引导加载程序 20

 4.1.3. Bootloader 镜像构成 20

 4.2. OpenSBI 开发指南 21

 4.2.1. OpenSBI 简介 21

 4.2.2. OpenSBI 下载与编译 21

 4.3. U-Boot 开发指南 21

 4.3.1. 目录结构解析 22

 4.3.2. U-Boot 下载与编译 22

 4.4. 高级定制与扩展 23

 4.4.1. 板级适配实战 23

 4.4.2. 启动画面定制 (LOGO) 24

 4.4.3. 添加自定义 U-Boot 命令 24

 4.4.4. U-Boot 升级脚本 25

5. Linux Kernel 开发指南 28

 5.1. 编译与构建流程 28

 5.1.1. 编译策略：全量与增量 28

 5.1.2. Debian 系统内核包 28

 5.1.3. Ubuntu 系统内核包 28

 5.1.4. 内核部署与升级 28

 5.2. 内核配置体系 28

 5.2.1. 平台配置对照表 28

 5.2.2. 定制化内核配置 29

 5.2.3. 高级定制与扩展 29

 5.3. 设备树 Overlay 30

 5.3.1. 添加新的 Overlay DTS 30

- 5.3.2. 编译与部署 30
 - 5.3.3. 平台差异配置 30
- 5.4. 驱动开发指引 31
- 6. 系统开发指南 32**
 - 6.1. 内存管理：MMZ 配置调整 32
 - 6.1.1. 核心约束与硬件信息 32
 - 6.1.2. 查看当前内存布局 32
 - 6.1.3. 在 U-Boot 中调整 MMZ 大小 32
 - 6.1.4. 验证配置生效 33
 - 6.2. 功耗调节与热管理 33
 - 6.2.1. 功耗调节整体方案 33
 - 6.2.2. 芯片温度监测 33
 - 6.2.3. 驱动层功耗调节方法 34
 - 6.2.4. 便捷工具：es_clk 脚本 35
 - 6.2.5. 硬件监控与风扇控制 35
 - 6.2.6. 高温保护流程建议 36

插图目录

1	HiFive Premier P550 外观	9
2	EBC77 系列单板计算机外观	11
3	EIC7700FG_EVB_LP5_A2 外观	12
4	EBC7702 Mini-DTX 外观	14
5	启动流程	20
6	业务场景下功耗调节示意	33

表格目录

1	P550 关键硬件参数	9
2	SBC 关键硬件参数	11
3	EIC7700FG_EVB_LP5_A2 关键硬件参数	13
4	EBC7702 Mini-DTX 关键硬件参数	14

1. 概述

本文档是 EIC77 系列 RISC-V AI SoC 平台的 BSP（Board Support Package，板级支持包）快速入门指南。旨在为底层软件开发人员提供一套标准化、系统化的开发指引，覆盖从环境搭建、源码编译到系统部署及底层功能定制的全生命周期。

EIC77 系列芯片基于高性能 RISC-V 架构设计，集成了强大的 AI 算力与丰富的工业级外设接口。本指南将作为核心参考，指导开发者如何结合具体的开发板硬件规格书与软件开发套件（SDK），高效完成以下任务：

- 计算平台开发：针对 EIC7700（单 Die）与 EIC7702（双 Die）平台的差异化特性进行开发。
- 系统镜像构建：掌握 Debian 与 Ubuntu 系统镜像的编译策略（全量与增量）及部署流程。
- 底层固件定制：深入理解 Bootloader（OpenSBI + U-Boot）的启动架构、移植方法与升级机制。
- 内核深度开发：进行 Linux Kernel 的配置裁剪、设备树（DTS/Overlay）适配及驱动集成。
- 系统级调优：实现内存管理（MMZ）、功耗热管理（DVFS）及硬件状态监控。

通过本指南，开发者将从“开箱”到“交付”，快速建立起对 EIC77 平台软硬件架构的完整认知与实操能力。

2. 硬件平台介绍

本章将详细介绍基于 EIC77 系列 SoC 的各类硬件开发平台。这些平台覆盖了从高性能桌面开发、边缘计算部署到紧凑型嵌入式集成的全场景需求，旨在为开发者提供多样化的硬件选型参考。

2.1. EIC7700 硬件平台

EIC7700 芯片定位于高性能 RISC-V 计算，配套平台兼顾了强大的 AI 算力与丰富的工业级接口。

2.1.1. HiFive Premier P550

HiFive Premier P550 开发板（简称 P550 或 DVB，因核心板包含 314 Pin，又称 D314）是一款面向下一代高性能计算与 AIoT 应用的高性能开发与评估平台。它基于奕斯伟计算（ESWIN）推出的 EIC7700 AI SoC，集成了高性能 SiFive RISC-V CPU 核心与强大的 AI 加速单元。

P550 采用创新的模块化设计，由 SOM 核心板（P550S）与 Carrier 载板（P550C）两部分组成：

- **P550S (核心板)**：集成了包括 EIC7700 SoC、高速 LPDDR5 内存、eMMC 存储、以太网 PHY 及电源管理系统在内的完整最小系统，是平台的“大脑”。
- **P550C (载板)**：作为系统的“骨架”，提供了稳定供电、全面的外部接口扩展以及丰富的调试支持。

这种设计不仅保证了核心计算单元的性能与可靠性，也通过载板提供了极致的接口灵活性与可扩展性。平台严格遵循 Mini-DTX 工业规格（203 mm × 170 mm），既可作为桌面开发利器，也能轻松集成到标准的 1U 服务器机箱中，无缝适应从实验室研发到边缘部署的全场景需求。

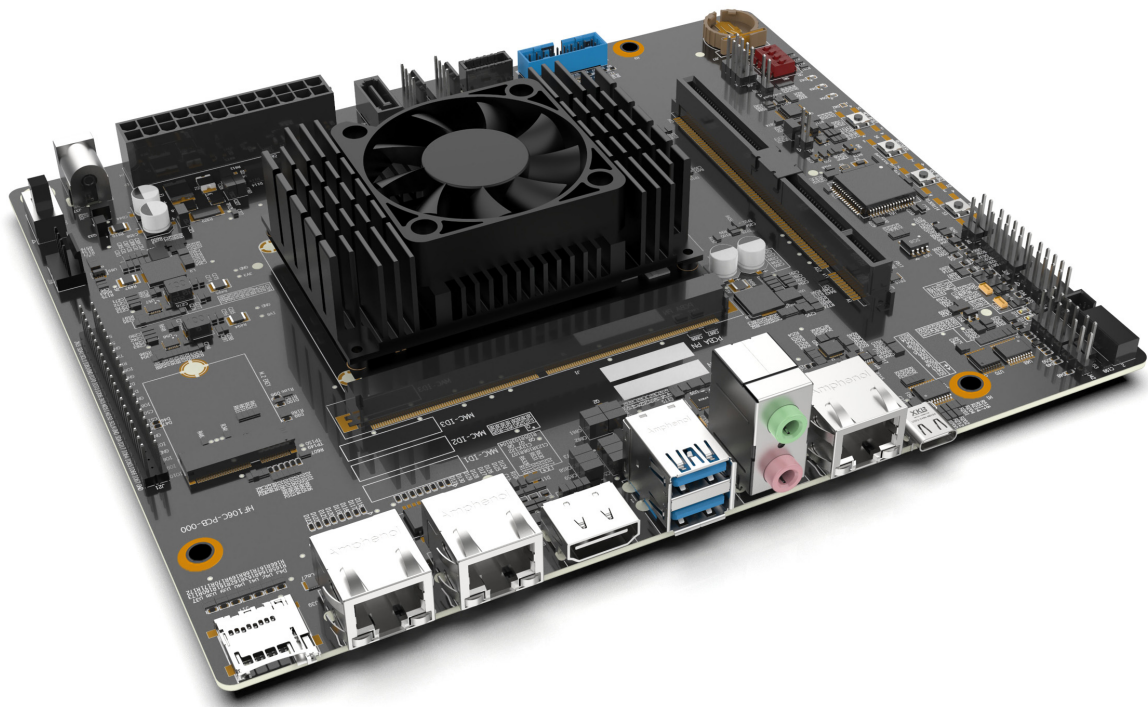


图 1: HiFive Premier P550 外观

表 1: P550 关键硬件参数

参数项	说明
外形尺寸	203 mm × 170 mm
CPU	64 位 OoO RISC-V 四核处理器
AI 处理器	最高 13.3 TOPS，支持 INT8/INT16/FP16
内存	板载 16 GB/32 GB LPDDR5 @ 6400 Mbps
存储	1× 板载 128 GB eMMC
	1× 16 MB SPI Flash
	1× 用于 SSD 的 SATA3.0 接口
	1× Micro SD 卡槽
	1× 板载 2 Kbit EEPROM
网络	2× 千兆以太网 RJ45 接口
	1× 百兆以太网 RJ45 接口
	1× 用于 Wi-Fi 模块的 M.2 Key E 接口

表 1: P550 关键硬件参数

参数项	说明
显示	1× 支持 4k@60 的 HDMI 2.0 Type-A 接口
	1× MIPI DSI 4-lane FPC 接口
摄像头	4× MIPI CSI 2-lane 或 2× MIPI CSI 2+2-lane
音频	1× 模拟音频接口
	1× F_audio 引脚输出阵列
PCIe	1× PCIe X16 插槽的 PCI Express Gen3 4-lane
USB	2× 标准 USB 3.2 Gen1 Type-A 接口
	1× USB 3.0 19-PIN 公头接口
	1× Type-E 接口
RTC	1× CR1220 RTC 电池座（电池不包含）
风扇	2× 2.54 mm 4-PIN 12V 风扇接口
	1× 2.54 mm 3-PIN 5V 风扇接口
按钮	1× SoC Reset 按钮
	1× Power 按钮
	1× MCU Reset 按钮
	1× Recovery 按钮
电源供应	1× 24-PIN ATX 电源接口 或 1× DC 2.5 mm 接口 (12V, 5A)
调试	1× 用于调试 UART 和 JTAG 的 USB Type-C 接口
其他 I/O	I ² C、I ² S、SPI、UART、通用 I/O 端口（映射到 40 针接口）

2.1.2. EBC77 系列单板计算机 (SBC)

EBC77 系列单板计算机（以下简称 SBC）是一款面向边缘 AI 计算、多媒体处理和紧凑型嵌入式应用的高性能、高集成度计算平台。它基于 EIC7700 AI SoC，集成了高性能 SiFive RISC-V CPU 核心与强大的 AI 加速单元。

SBC 采用创新的高密度单板化设计，在仅有 85 mm × 56 mm 的极小板面上，集成了包括 CPU、内存、存储、丰富的多媒体与高速接口在内的完整系统。这种设计摒弃了模块化分离，实现了极致的紧凑性，是构建空间受限的智能终端、边缘盒子、AI 摄像机和工业控制设备的理想“心脏”。

SBC 凭借其旗舰级硬件配置、超小型封装和全面的接口支持，能够无缝集成到各种严苛空间和形态的终端产品中，为从原型验证到规模化部署提供强大、可靠的硬件基石。



图 2: EBC77 系列单板计算机外观

表 2: SBC 关键硬件参数

参数项	说明
外形尺寸	85 mm × 56 mm
CPU	64 位 OoO RISC-V 四核处理器
AI 处理器	最高 13.3 TOPS，支持 INT8/INT16/FP16
内存	板载 8 GB/16 GB/32 GB LPDDR5 @ 6400 Mbps
存储	1× 8 MB SPI Flash
	1× Micro SD 卡槽
网络	1× 千兆以太网 RJ45 接口
	1× 板载 802.11ac 双频 Wi-Fi 模块
显示	1× Micro HDMI 2.0 输出
	1× 4-lane MIPI DSI TX（兼容 MIPI CSI RX）
摄像头	1× 4-lane MIPI CSI RX（兼容 MIPI DSI TX）
	1× 4-lane MIPI CSI RX
音频	通过 40-PIN 扩展接口

表 2: SBC 关键硬件参数

参数项	说明
PCIe	1× 4-lane PCIe Gen3.0 FPC 连接器
USB	2× USB 3.0 Type-A 接口 (堆叠)
	2× USB 2.0 Type-A 接口 (堆叠)
RTC	1.0 mm 2-PIN 3V 连接器
风扇	1.0 mm 4-PIN 5V 连接器
按键	1× 复位键
	1× 启动选择键
电源供应	USB Type-C PD, 推荐 ≥ 45 W (15 V@3 A)
调试	1× Micro-B 调试端口
其他 I/O	I ² C、I ² S、SPI、UART、通用 I/O 端口 (映射到 40 针扩展接口)

2.1.3. EIC7700FG EVB LP5

EIC7700FG EVB LP5 (简称 EVB) 是一款基于奕斯伟计算 (ESWIN) EIC7700 AI SoC 的高性能、全功能开发评估板。该平台集成了旗舰级 RISC-V CPU、强大的 AI 加速单元、高速 LPDDR5 内存及丰富的工业级外设接口, 是面向边缘 AI 计算、机器视觉、高端嵌入式系统、网络应用等领域进行原型开发、算法验证和性能评估的理想硬件平台。

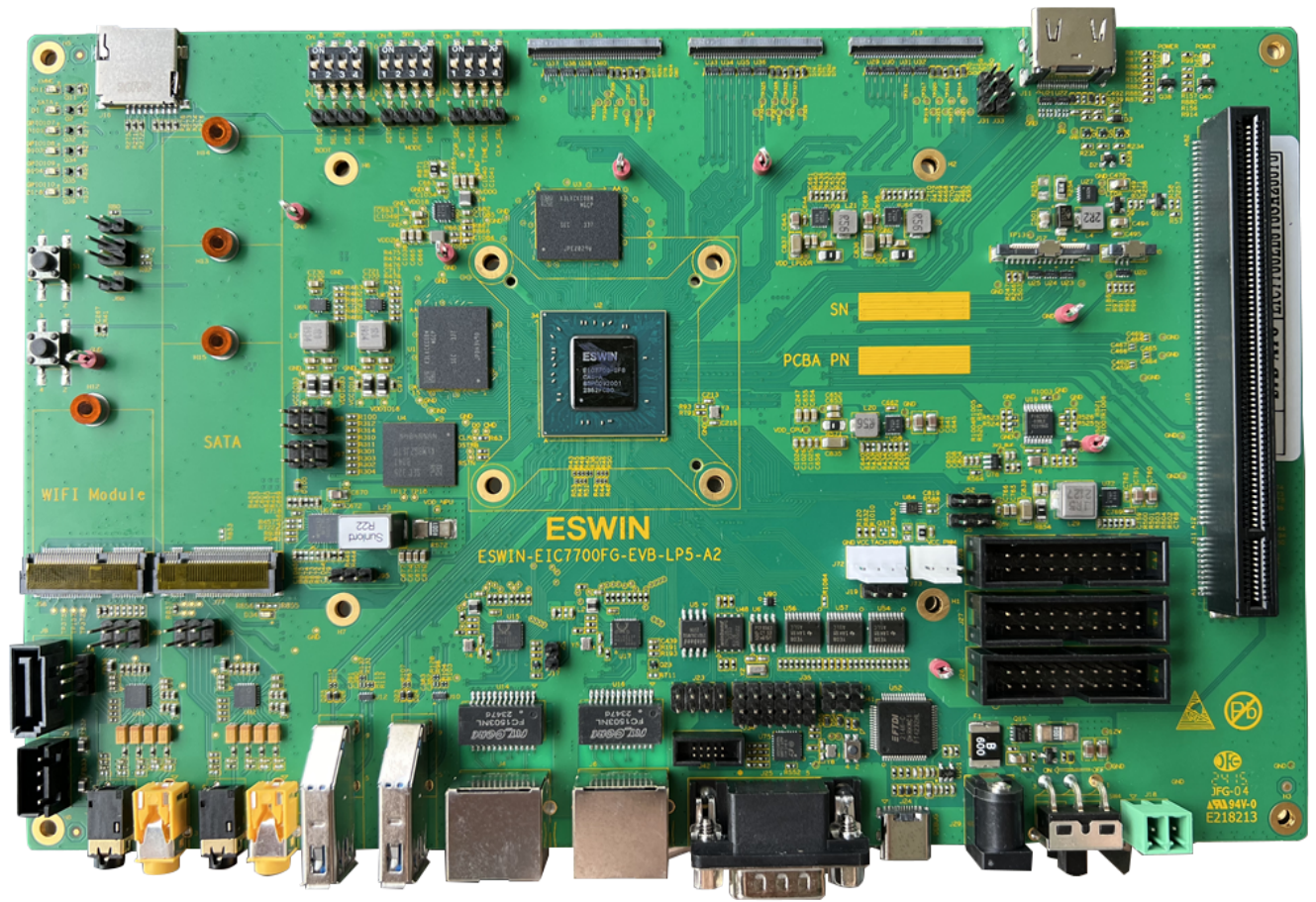


图 3: EIC7700FG_EVB_LP5_A2 外观

表 3: EIC7700FG_EVB_LP5_A2 关键硬件参数

参数项	说明
板卡规格	150.04 mm × 235 mm
CPU	64 位 乱序执行 RISC-V 四核处理器
AI 处理器	算力最高 13.3 TOPS，支持 INT8/INT16/FP16 精度
内存	板载 16 GB/32 GB LPDDR5，速率 6400 Mbps
存储	1× 板载 32 GB eMMC
	1× 板载 16 MB SPI Flash
	1× M.2 Key B&M 连接器（用于 SSD）
	1× Micro SD 卡槽
网络	2× 千兆以太网 RJ45 接口
	1× M.2 Key E 连接器（用于 Wi-Fi/蓝牙模块）
显示	1× HDMI 2.0 Type-A 接口，支持 4k@60
	1× MIPI DSI 4-lane FPC 连接器
摄像头	3× MIPI CSI 2+2-lane 或 4-lane FPC 连接器
音频	2× 3.5 mm 音频输出接口
	2× 3.5 mm CTIA 标准耳机接口
PCIe	1× PCI Express Gen3 x4 lane（通过 PCIe X16 插槽）
USB	2× 标准 USB 3.2 Gen1 Type-A 接口
风扇	1× 2.54 mm 4-针 5 V 风扇接口
	1× 2.54 mm 3-针 5 V 风扇接口
按键	1× 复位键
	1× 启动选择键
电源供应	1× DC 2.5 mm 接口 (12 V, 5 A)
	1× 电源滑动开关（开/关）
	1× 远程电源开/关连接器
调试	1× USB Type-C 接口（用于调试串口和 JTAG）

2.2. EIC7702 硬件平台

EIC7702 芯片作为 EIC7700 的进阶版本，引入了双 Die 架构（Dual-Die），在保持 Mini-DTX 规格的同时，大幅提升了计算密度与 AI 算力。

2.2.1. EBC7702 Mini-DTX

EBC7702 Mini-DTX（简称 EBC7702，因核心板包含 560 Pin，又称 D560）基于奕斯伟计算（ESWIN）最新推出的 EIC7702 AI SoC，该芯片集成了高性能 64 位 RISC-V 多核 CPU 与强大的自研 NPU 加速单元，在 Mini-DTX 规格的板型内实现了卓越的计算密度。

EBC7702 采用创新的模块化设计，由 SOM 核心板（D560S）与 Carrier 载板（D560C）两部分组成：

- **D560S(核心板)**: 集成了包括 EIC7702 SoC、板载 LPDDR5 内存、eMMC 存储、千兆以太网 PHY 及核心电源管理系统在内的完整最小系统，是整个平台的“大脑”。
- **D560C(载板)**: 作为系统的“骨架”，提供 ATX/DC 双供电输入、丰富的外部接口（PCIe、USB、HDMI、MIPI 等）以及完善的调试支持，确保系统在各类应用场景下的可扩展性与可靠性。

本主板严格遵循 Mini-DTX 工业标准规格（203 mm × 170 mm），兼具桌面开发的灵活性与工业部署的稳定性，既可

置于办公环境作为开发评估平台，也可直接集成到标准 1U 服务器机箱或嵌入式设备中，满足从实验室原型验证到边缘端大规模部署的全生命周期需求。

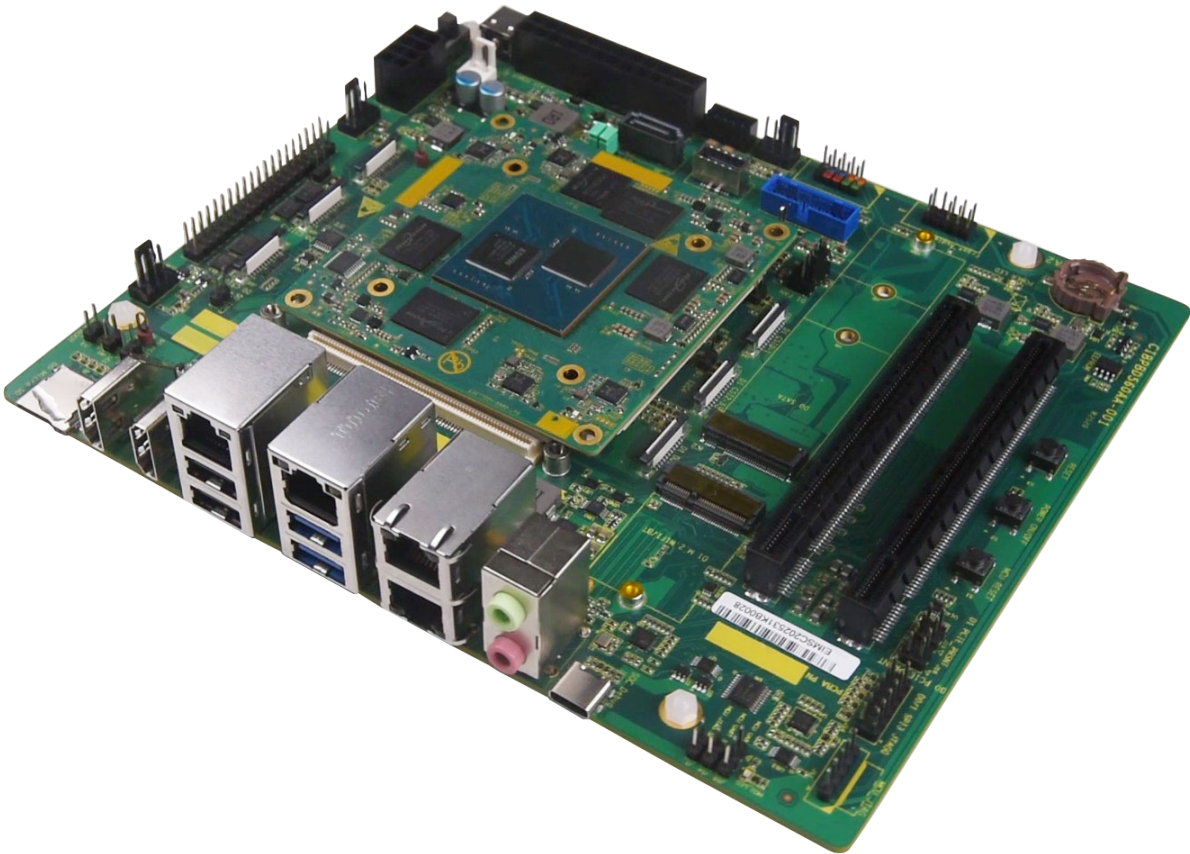


图 4: EBC7702 Mini-DTX 外观

表 4: EBC7702 Mini-DTX 关键硬件参数

参数项	规格说明
外形规格	203 mm × 170 mm
中央处理器	64 位乱序执行 RISC-V RV64GC 八核处理器
AI 处理器	最高 26.6 TOPS，支持 INT8/INT16/FP16
内存	板载 32 GB/64 GB LPDDR5，速率 6400 Mbps
存储	1× 板载 32 GB eMMC 存储器
	2× 板载 16 MB SPI 闪存
	1× M.2 M-Key SATA 接口插槽
	1× Micro SD 卡插槽
	1× 板载 2 Kbit EEPROM 存储器
网络	4× 千兆以太网 RJ45 接口（其中一个支持局域网唤醒功能）
	1× M.2 E-Key Wi-Fi 模块接口（通过 SDIO/UART 连接）

表 4: EBC7702 Mini-DTX 关键硬件参数

参数项	规格说明
显示	2× HDMI 输出接口
	2× 4-lane MIPI DSI 接口插槽
摄像头	4× 4-lane MIPI CSI 接口插槽
音频	1× 音频输入接口
	1× 音频输出接口
	1× 前面板音频线路输入/输出信号排针接口
PCIe 扩展	2× 4-lane PCIe Gen3.0 x16 插槽
USB 接口	2× USB 3.0 Type-A 接口（堆叠式）
	2× USB 2.0 Type-A 接口（堆叠式）
	1× USB 19-针接口（用于支持前面板 1 个 USB 3.0 Type-A 接口）
	1× USB Type-E 接口（用于支持前面板 1 个 USB 3.0 Type-C 接口）
电源供应	ATX 电源接口
调试接口	1× USB 2.0 Type-C 端口（用于板载 MCU 调试或 SoC 调试，默认配置）
实时时钟	1× CR1220 RTC 电池座（电池不包含）
风扇接口	4× 2.54 毫米 4-针 12V 风扇接口
其他输入输出接口	I ² C、I ² S、SPI、UART、通用输入输出端口（映射到 40 针排针接口）

2.3. 硬件平台用户手册

本节汇总了 EIC77 系列 SoC 硬件平台的官方用户手册。这些文档包含了详细的硬件规格、接口定义及使用指南，是硬件设计、系统集成与二次开发的核心依据。请根据您的目标平台，点击下方链接查看完整文档。

板卡型号	用户手册
HiFive Premier P550	HiFive_Premier_P550 用户手册
EBC77 SBC	EBC77 系列单板计算机 SBC 用户手册
EIC7700FG EVB LP5	EIC7700FG_EVB_LP5_A2 用户手册
EBC7702 Mini-DTX	EBC77 系列 EBC7702_Mini-DTX 用户手册

3. 系统编译指南

本章将详细介绍如何在 PC 端搭建 EIC77 系列 SoC 的完整系统编译环境，涵盖 U-Boot、OpenSBI、Linux Kernel 及 Debian/Ubuntu 系统镜像的构建流程。

3.1. 编译环境说明

- 在开始编译前，请确保您的开发主机满足以下基础环境要求：
- 操作系统：推荐 Ubuntu 22.04 LTS。对于 Ubuntu 20.04 用户，建议通过官方提供的 Docker 容器环境进行编译。
- 硬件架构：x86_64
- 网络配置：编译过程中需要从 GitHub、Ubuntu Ports 等代码托管平台拉取源码，请确保网络通畅。

3.2. Ubuntu22.04 原生编译环境搭建

本节适用于直接在 Ubuntu 22.04 上进行编译的场景。

3.2.1. 安装通用编译依赖

```
sudo apt install -y \
  gdisk dosfstools build-essential libncurses-dev gawk flex bison \
  openssl libssl-dev tree dkms libelf-dev libudev-dev libpci-dev \
  libiberty-dev autoconf device-tree-compiler xz-utils devscripts \
  ccache debhelper wget curl libconfuse-dev mtools fastboot rsync \
  debootstrap mmdebstrap qemu-user-static binfmt-support qemu-utils \
  germinate git snapd python3 python3-pip ubuntu-dev-tools
```

提示

Ubuntu22.04 默认安装 python3.10，在编译 ubuntu 镜像时需要安装 tomli 库。

```
# 安装 tomli 库
pip3 install tomli
```

3.2.2. 安装 RISC-V 交叉编译工具链

1. 获取 RISC-V 交叉编译工具链

点击下载 RISC-V 交叉编译工具链 [riscv64-glibc-ubuntu-22.04-gcc-nightly-2024.04.12-nightly.tar.gz](https://github.com/riscv/riscv-gnu-toolchain/releases/download/2024.04.12-nightly/riscv64-glibc-ubuntu-22.04-gcc-nightly-2024.04.12-nightly.tar.gz)

2. 解压到/opt 目录下：

```
sudo tar -xvf riscv64-glibc-ubuntu-22.04-gcc-nightly-2024.04.12-nightly.tar.gz -C /opt
```

3. 配置环境变量（写入 ~/.bashrc）：

```
echo 'export PATH=/opt/riscv/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
```

4. 验证工具链：

```
riscv64-unknown-linux-gnu-gcc -v
```

3.3. Docker 编译环境搭建

可直接使用 ESWIN 提供的 Docker 容器以获得一致的编译环境。

1. 安装 Docker Engine

```
sudo apt update
sudo apt install -y apt-transport-https ca-certificates curl software-properties-common
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
  ↳ /usr/share/keyrings/docker-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-archive-keyring.gpg]
  ↳ https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee
  ↳ /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
sudo apt install -y docker-ce
```

2. 加载官方 Docker 编译镜像

获取官方提供的 es_compile_docker.tar 镜像文件，并执行加载：

```
sudo docker load -i es_compile_docker.tar
```

3. 配置多架构支持（仅 Ubuntu 20.04 需要）

若主机使用 Ubuntu 20.04 系统，必须执行以下命令注册 QEMU 静态模拟器，否则编译会出错。Ubuntu 22.04 及以上版本通常不需要执行。

```
sudo docker pull multiarch/qemu-user-static
sudo docker run --rm --privileged multiarch/qemu-user-static --reset -p yes
```

3.4. Debian 系统编译

1. 获取 SDK 并解压

从 ESWIN 官方网站[开发者社区](#)下载最新的 Debian 系统镜像编译 sdk 包 `eswin-sdk-*-debian.tar.gz`，并解压至用户目录：

```
mkdir -p $HOME/workspace
tar -xzf eswin-sdk-*-debian.tar.gz -C $HOME/workspace
cd $HOME/workspace
```

2. （可选）提前下载源码

若网络环境不佳，可提前克隆指定版本的源码至 source/目录。

版本号获取：请通过以下命令从 setenv.sh 中获取当前 SDK 对应的 Release Tag。

```
head setenv.sh | grep "RELEASE_TAG"
```

手动下载命令示例：

```
git clone --depth=1 -b EIC7X-XXXX https://github.com/eswincomputing/u-boot.git source/u-boot-eswin
git clone --depth=1 -b EIC7X-XXXX https://github.com/eswincomputing/opensbi.git source/opensbi-eswin
git clone --depth=1 -b EIC7X-XXXX https://github.com/eswincomputing/linux-stable.git source/linux-eswin
```

提示

请将 EIC7X-XXXX 替换为实际获取的版本号。

3. 启动编译环境

- Docker 环境：

```
sudo docker run --rm --privileged -it -v $HOME/workspace:/home/workspace -u root -w  
↪ "/home/workspace" es_compile
```

- Ubuntu 22.04 (原生环境)：直接进入 SDK 目录即可。

4. 执行编译

在 SDK 根目录下执行以下命令初始化环境，根据提示选择目标板卡：

```
source setenv.sh

board list:
[1] eic7700-evb #EIC7700FG EVB
[2] eic7700-hifive-premier-p550
[3] eic7702-d560 #EBC7702_Mini-DTX Mainboard
[0] Exit
please select[0-3]:2 #以 2 编译 Hifive-Premier-P550 为例

you chose eic7700-hifive-premier-p550
Use the following method to start compiling:
  make_bootchain #编译 bootloader, 生成 bootloader_*.bin 和 recovery_bootloader_*.bin
  make_kernel #编译 kernel, 生成 linux-*.deb 包
  make_debug_kernel
  make_desktop_images #生成 root.ext4 和 boot.ext4 镜像, 需要保证 kernel 已经编译完成
  make_all:bootchain kernel desktop_images #同时编译 bootloader、kernel、desktop_images
```

执行全编译

```
make_all
```

编译完成后可看到编译生成的 `bootloader_bin`、`root.ext4`、`boot-*.ext4` 等镜像文件，烧写方法参照各板卡用户手册。

提示

编译过程中制作 images 这个流程需要 `sudo` 权限，需要输入一次 `sudo` 的密码，使用 `docker` 镜像不会提示输入密码。

3.5. Ubuntu 系统编译

1. 获取 SDK 并解压

从 ESWIN 官方网站[开发者社区](#)下载最新的 Ubuntu 镜像编译 sdk 包 `eswin-sdk-*-ubuntu.tar.gz`，并解压至用户目录：

```
mkdir -p $HOME/workspace
tar -xf eswin-sdk-*-ubuntu.tar.gz -C $HOME/workspace
cd $HOME/workspace
```

2. SDK 目录结构

- `setenv.sh`：编译环境初始化脚本
- `README`：编译说明
- `source/`：source 目录说明

目录名称	功能说明
firmware-eswin	EIC7700/7702 系列 SoC 专用固件
linux-eswin	EIC7700/7702 系列定制化 Linux 内核源码
opensbi-eswin	适配 EIC7700/7702 平台的 OpenSBI 源码
risc-v-gadget	RISC-V 架构专用的工具与配置
uboot-eswin	EIC7700/7702 系列定制化 U-Boot 源码
ukpack	系统镜像打包工具

提示

编译过程会自动从 GitHub 拉取内核、Bootloader 等源码，若网络不稳定导致下载失败，参考 Debian 系统编译中的手动下载方法。

3. 启动编译环境

- Docker 环境：

```
sudo docker run --rm --privileged -it -v $HOME/workspace:/home/workspace -u root -w
↪ "/home/workspace" es_compile
```

- Ubuntu 22.04 (原生环境)：直接进入 SDK 目录即可。

4. 执行编译

在 SDK 根目录下执行以下命令初始化环境，根据提示选择目标板卡：

```
source setenv.sh
board list:
[1] eic7700-hifive-premier-p550
[2] eic7702-d560 #EBC7702_Mini-DTX Mainboard
[3] eic7700-sbc
[0] Exit
please select[0-3]:1 #以 1 编译 Hifive-Premier-P550 为例

you chose eic7700-hifive-premier-p550
Use the following method to start compiling:
make_bootchain #编译 bootloader, 生成 bootloader_*.bin 和 recovery_bootlader_*.bin
make_kernel #编译 kernel, 生成 linux-*.deb 包
make_images #生成一个包含 kernel 和 rootfs 的 ubuntu 镜像, 需要保证 kernel 已经编译完成
make_all:bootchain kernel images #编译整个 ubuntu 镜像, 包括 bootloader、kernel、rootfs
```

执行全编译

```
make_all
```

编译完成后可看到编译生成的 `bootloader_bin`、`ubuntu.img` 等镜像文件，烧写方法参照各板卡用户手册。

4. Bootloader 开发指南

本章节针对 EIC77 系列 SoC 平台，系统化梳理 U-Boot 与 OpenSBI 的源码获取、编译、移植、定制化开发全流程，旨在为开发者提供标准化的移植指导和深度开发参考。

提示

EIC77 系列 SoC Bootloader 体系采用 **OpenSBI + U-Boot** 的两级启动架构。其中，OpenSBI 作为 RISC-V 标准的 M 模式接口层，通常无需修改；开发者主要聚焦于 U-Boot 进行功能扩展与硬件适配。

4.1. 启动架构与 Bootloader 概述

4.1.1. Linux 系统启动流程

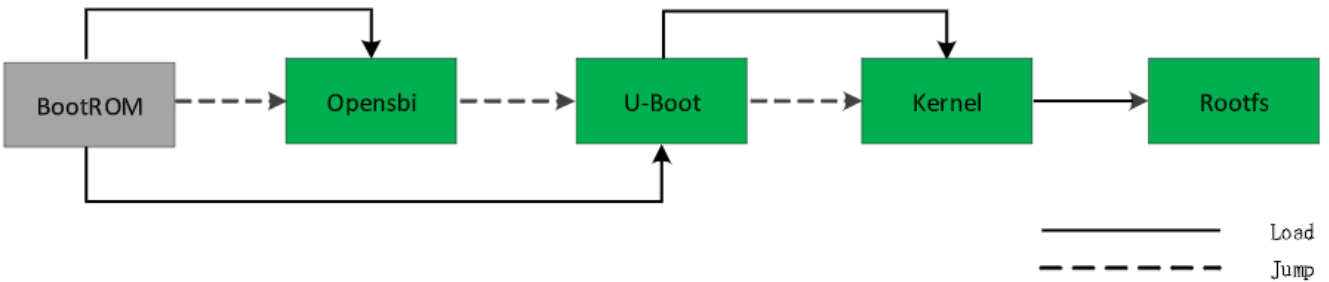


图 5: 启动流程

Linux 系统启动流程如上图所示，包含以下关键模块：

1. **BootROM (SCPU)** → 加载并验签 OpenSBI+U-Boot；
2. **OpenSBI (M 模式)** → 为 S 模式提供标准 SBI 接口；
3. **U-Boot (S 模式)** → 加载内核、环境变量管理、固件升级；
4. **Linux Kernel** → 驱动初始化、启动用户态应用；
5. **Rootfs** → 用户程序与运行库。

4.1.2. 二级引导加载程序

EIC77 系列 SoC 采用二级启动架构，协同工作如下：

1. **一级启动 (BootROM)**：固化在芯片 ROM 中，不可修改。负责最基础的硬件初始化、安全校验，并加载二级 Bootloader。
2. **二级启动 (OpenSBI + U-Boot)**：运行在主核 (MCPU)，负责加载 Linux 内核、设备树，并提供固件升级与恢复功能。

提示

EIC77 系列 SoC 内部包含三种不同职责的 RISC-V 核心：

- **MCPU (U84)**：主核，运行 OpenSBI、U-Boot 及 Linux Kernel。
- **SCPU (E31)**：安全核，运行 BootROM、安全固件 (`dieX_sec_fw.bin`) 及 DDR 固件 (`ddr_fw.bin`)。
- **LPCPU (E31)**：低功耗核，运行独立的功耗管理固件。

4.1.3. Bootloader 镜像构成

Bootloader 并非单一文件，而是通过 `nsign` 工具打包的**组合镜像包**，主要包含：

镜像名称	功能描述
die0_sec_fw.bin / die1_sec_fw.bin	核心安全固件
ddr_fw.bin	DDR 初始化与训练固件
die0_lpcpu_load_fw.bin	低功耗核加载固件
fw_payload.bin	OpenSBI + U-Boot 合并镜像

4.2. OpenSBI 开发指南

4.2.1. OpenSBI 简介

OpenSBI (RISC-V Open Source Supervisor Binary Interface) 是 RISC-V 架构的**标准二进制接口**，运行在 M 模式。它为 S 模式 (U-Boot/Kernel) 屏蔽了底层硬件差异，提供统一的系统服务接口。

4.2.2. OpenSBI 下载与编译

本节仅简单介绍 OpenSBI 的源码获取、编译过程，参考[系统编译指南](#)一章中的 `make_bootchain` 命令来编译生成整个 bootloader 镜像。

源码获取：

```
git clone --branch opensbi-1.3-EIC7X https://github.com/eswincomputing/opensbi.git
```

编译前置条件：

- 1. 已编译 U-Boot，生成 `u-boot.bin` 和 `u-boot.dtb`；
- 2. 已配置 RISC-V 交叉编译器，参考[安装 RISC-V 交叉编译工具链](#)。

编译命令：

```
make PLATFORM=eswin/eic770x \
  FW_PAYLOAD=y \
  FW_PAYLOAD_PATH=u-boot.bin \
  FW_FDT_PATH=u-boot.dtb \
  CHIPLET="BR2_CHIPLET_1" \
  CHIPLET_DIE_AVAILABLE="BR2_CHIPLET_1_DIE0_AVAILABLE" \
  MEM_MODE="BR2_MEMMODE_FLAT" \
  PLATFORM_CLUSTER_X_CORE="BR2_CLUSTER_4_CORE" \
  ENABLE_VPU_SDK=0 ENABLE_ECC=0 \
-j8
```

清理编译产物：

```
make clean
```

编译产物：

```
build/platform/eswin/eic770x/firmware/fw_payload.bin
```

4.3. U-Boot 开发指南

在 EIC77 系列 SoC 平台的系统启动流程中，U-Boot (Universal Boot Loader) 扮演着至关重要的角色。作为系统启动过程中的二级引导加载程序，U-Boot 承接了从 OpenSBI (M 模式) 到 Linux Kernel (S 模式) 的关键过渡。

本章旨在帮助开发者快速掌握在 EIC77 系列 SoC 平台上进行 U-Boot 开发的核心技能。无论您是需要进行板级适配、驱动开发，还是启动流程定制，本章都将为您提供清晰的路径。

4.3.1. 目录结构解析

目录/文件	功能描述
arch	CPU 架构相关代码（RISC-V 核心实现）
board	开发板级定制化代码（如 P550、EBC7702）
cmd	U-Boot 命令实现源码
common	通用核心功能代码
drivers	设备驱动（UART/I2C/SPI/ETH/SD 等）
dts	设备树编译配置（Kconfig/Makefile）
env	环境变量管理代码
fs	文件系统支持（ext4/fat 等）
include	全局头文件
lib	通用库文件
net	网络协议栈相关
scripts	编译/配置脚本
Makefile	主编译脚本
.config	编译后生成的最终配置文件

4.3.2. U-Boot 下载与编译

本节仅简单介绍 U-Boot 的源码获取、编译过程，参考[系统编译指南](#)一章中的 `make_bootchain` 命令来编译生成整个 bootloader 镜像。

源码获取：

```
git clone https://github.com/eswincomputing/u-boot.git --branch u-boot-2024.01-EIC7X
```

配置 RISC-V 交叉编译器：

参考[安装 RISC-V 交叉编译工具链](#)。

编译 U-Boot：

```
export ARCH=riscv
export CROSS_COMPILE=riscv64-unknown-linux-gnu-

# 加载默认配置 (以 eic7700_evb_a2 为例)
make eic7700_evb_a2_defconfig

# 编译 (-j8 加速编译)
make -j8

# 清理编译产物
make clean
```

编译产物：

根目录下的 `u-boot.bin` 和 `u-boot.dtb`。

4.4. 高级定制与扩展

4.4.1. 板级适配实战

本节从零开始演示如何基于现有模板（如 eic7700_evb_a2）创建自定义开发板（test_board）配置：

1. 添加 **Kconfig** 配置 (arch/riscv/Kconfig)：

```
config TARGET_TEST_BOARD
    bool "Support Eswin EIC7700 test Board"
    ...
    source "board/eswin/test_board/Kconfig"
```

2. 创建并修改 **defconfig**：

```
cd configs
cp eic7700_evb_a2_defconfig test_board_defconfig
```

修改 test_board_defconfig：

```
CONFIG_DEFAULT_DEVICE_TREE="test_board"
CONFIG_TARGET_TEST_BOARD=y
```

3. 添加设备树文件：

```
cd arch/riscv/dts
cp eic7700_evb_a2.dts test_board.dts
```

修改 Makefile：

```
dtb-$(CONFIG_TARGET_TEST_BOARD) += test_board.dtb
```

4. 创建板级目录与代码：

```
cd board/eswin
cp -r eic7700_evb test_board
cd test_board
mv eic7700.c test_board.c
```

修改 Makefile：

```
obj-y += test_board.o
```

修改 Kconfig，替换目标配置与板级名称。

```
if TARGET_ESWIN_EIC7700 # 修改为 TARGET_TEST_BOARD
config SYS_BOARD
    default "eic7700_evb" # 修改为 "test_board"

config SYS_CONFIG_NAME
    default "eswin_eic7700_evb" # 修改为 "test_board"
```

5. 添加板级头文件：


```
cd include/configs
cp eswin_eic7700_evb.h test_board.h
```

6. 编译新 Board:

```
make test_board_defconfig
make -j8
```

编译通过后将会在 u-boot 目录下生成 u-boot.bin 和 u-boot.dtb

4.4.2. 启动画面定制 (LOGO)

支持 **Preboot (DDR 阶段)** 和 **U-Boot 阶段** LOGO 定制, 支持 BMP 格式。

命令行烧写 LOGO:

```
# 1. 准备图片
gzip -c logo.bmp > logo.bmp.gz

# 2. 加载到内存
ext4load usb 0 0x100000000 logo.bmp.gz

# 3. 烧写 Preboot LOGO (可指定坐标)
es_burn preboot_logo 0x100000000 0 0

# 4. 烧写 U-Boot LOGO
es_burn uboot_logo 0x100000000
setenv splashpos 0,0
saveenv

# 5. 如需清除自定义开机 LOGO、恢复出厂默认画面, 只需擦除 Flash 中存储 LOGO 的专用分区, 无需重刷完整固件。擦
    除后重启, 设备将自动回退至原厂默认开机画面
sf probe 0
bootspi probe 0
bootspi wp 0
sf erase 0xe80000 0x80000 # 清空 Preboot
sf erase 0xd00000 0x180000 # 清空 U-Boot
```

代码内置 LOGO: 将压缩后的图片放入 tools/logos/test_board.bmp.gz, 并修改板级头文件:

```
#define CONFIG_EXTRA_ENV_SETTINGS \
    "splashfile=test_board.bmp.gz\0" \
    "splashpos=1660,0\0" \
    ...
```

4.4.3. 添加自定义 U-Boot 命令

命令核心数据结构:

```
struct cmd_tbl {
    char *name;           // 命令名称
    int maxargs;          // 最大参数个数
    int (*cmd)(...);      // 命令执行函数
    char *usage;          // 简短说明
    char *help;           // 详细帮助
};
```

命令定义与注册（以 `bootm` 为例）：

```
// 命令实现函数
int do_bootm(struct cmd_tbl *cmdtp, int flag, int argc, char *const argv[])
{
    // 业务逻辑
    return 0;
}

// 注册命令
U_BOOT_CMD(
    bootm, CONFIG_SYS_MAXARGS, 1, do_bootm,
    "boot application image from memory",
    "bootm [addr] - boot kernel from memory"
);
```

- `bootm` 是命令字符串
- `CONFIG_SYS_MAXARGS` 命令支持的最大参数
- `1` 表示重复一次
- `do_bootm` 表示对应的命令处理函数是 `do_bootm`
- “boot application image from memory” 表示的是 `usage` 字符串内容
- `bootm_help_text` 表示 `help` 字符串是 `bootm_help_text` 定义的字符串

4.4.4. U-Boot 升级脚本

U-Boot Script 是包含一系列 U-Boot 命令的文本文件，将需要升级的镜像文件放在脚本中，并使用 `mkimage` 工具编译为 U-Boot 可识别的镜像格式。

此方法适用于在工厂生产等场景，通过 SD 卡或 U 盘来一键烧录整个系统镜像。

升级脚本示例：

- `update-sd.txt`：从 **SD 卡** 升级

```
env save echo "Recover from SD ..."

if test -e mmc 1 partition
then
    echo "Ready to partition"
    ##### use default partition #####
    run gpt_partition
fi

if test -e mmc 1 bootchain.bin
then
    echo "Ready to burn bootchain"
    ext4load mmc 1 0x100000000 bootchain.bin
    es_burn write 0x100000000 flash
    echo "Burn bootchain successfully"
fi

if test -e mmc 1 boot.ext4
then
    echo "Ready to burn boot partition"
    es_fs update mmc 1 boot.ext4 mmc 0:1
fi

if test -e mmc 1 root.ext4
then
    echo "Ready to burn root partition"
```

```
es_fs update mmc 1 root.ext4 mmc 0:3
fi
```

- `update-usb.txt` : 从 **U 盘** 升级

```
env save echo "Recover from USB ..."

if test -e usb 0 partition
then
    echo "Ready to partition"
    ##### use default partition #####
    run gpt_partition
fi

if test -e usb 0 bootchain.bin
then
    echo "Ready to burn bootchain"
    ext4load usb 0 0x100000000 bootchain.bin
    es_burn write 0x100000000 flash
    echo "Burn bootchain successfully"
fi

if test -e usb 0 boot.ext4
then
    echo "Ready to burn boot partition"
    es_fs update usb 0 boot.ext4 mmc 0:1
fi

if test -e usb 0 root.ext4
then
    echo "Ready to burn root partition"
    es_fs update usb 0 root.ext4 mmc 0:3
fi
```

脚本核心逻辑：

1. 检测外部存储是否存在 `partition` 标记文件，若存在则对 eMMC 执行 GPT 分区；
2. 检测并烧录 `bootloader.bin` 到 SPI Flash；
3. 检测并更新 `boot.ext4` 到 eMMC boot 分区；
4. 检测并更新 `root.ext4` 到 eMMC root 分区；
5. 升级完成。

脚本编译：txt 脚本必须通过 `mkimage` 工具打包为 U-Boot 可识别的镜像格式，`mkimage` 位于 U-Boot 编译后的 `tools/` 目录下。

```
# 编译 U 盘升级脚本
mkimage -A riscv -O linux -T script -C none -a 0x90000000 -e 0x90000000 -n "U-Boot Script" -d
    ↪ update-usb.txt usbupdate.scr

# 编译 SD 卡升级脚本
mkimage -A riscv -O linux -T script -C none -a 0x90000000 -e 0x90000000 -n "U-Boot Script" -d
    ↪ update-sd.txt sdupdate.scr
```

升级介质准备：

1. 升级介质（U 盘 / SD 卡）格式化为 **EXT4**；
2. 根目录放入以下文件：
 - 升级脚本：`usbupdate.scr` / `sdupdate.scr`

- 启动镜像: `bootloader.bin`
- 系统分区: `boot.ext4`、`root.ext4`

手动升级步骤:

1. 插入制作好的 U 盘至开发板 **USB1** 接口, 各板卡 USB 接口位置参考对应用户手册;
2. 开发板上电, 进入 U-Boot 命令行;
3. 执行升级命令:
 - U 盘升级: `run usbupdate`
 - SD 卡升级: `run sdupdate`
4. 等待升级完成。

提示

若希望在设备启动时自动升级, 可以在 U-boot 启动中增加检测逻辑, 当检查到 U 盘和 SD 卡是否存在升级脚本时, 若存在则自动执行升级, 不存在则继续启动系统。仅提供思路, 有需要可联系 ESWIN FAE 提供支持。

5. Linux Kernel 开发指南

本章旨在为 EIC77 系列 SoC 平台的开发者提供 Linux Kernel 开发的全景式指引。内容将覆盖从源码获取、环境配置到内核调试的完整生命周期，适用于底层系统开发工程师及 BSP 维护人员。

5.1. 编译与构建流程

在完成[系统编译指南](#)中的环境配置和 SDK 下载后，即可开始 Kernel 开发。

5.1.1. 编译策略：全量与增量

- **全量构建 (`make_all`)**：Debian 系统的 kernel 包含在 `boot.ext4` 文件系统中，Ubuntu 系统的 kernel 包含在 `Ubuntu.img` 镜像中。生成这些完整镜像需要使用 `make_all` 命令，但该命令编译时间较长，**不利于快速调试**。
- **增量构建 (`make_kernel`)**：专为 Kernel 调试设计。该命令仅编译内核及相关模块，并生成可直接安装的 Debian 软件包（.deb），大幅提升迭代效率。

5.1.2. Debian 系统内核包

以 EIC7700 为例，执行 `make_kernel` 后生成的内核软件包如下（EIC7702 包名略有不同）：

```
linux-headers-6.6.115-eic7x-2025.12_6.6.115-**_riscv64.deb
linux-image-6.6.115-eic7x-2025.12_6.6.115-**_riscv64.deb
linux-libc-dev_6.6.115-**_riscv64.deb
```

5.1.3. Ubuntu 系统内核包

以 EIC7700 为例，执行 `make_kernel` 后生成的内核软件包如下（EIC7702 包名略有不同）：

```
linux-headers-6.6.115-2026-eic7700_6.6.115-**_riscv64.deb
linux-headers-eic7700_6.6.115-**_riscv64.deb
linux-image-6.6.115-2026-eic7700_6.6.115-**_riscv64.deb
linux-image-eic7700_6.6.115-**_riscv64.deb
linux-modules-6.6.115-2026-eic7700_6.6.115-**_riscv64.deb
```

5.1.4. 内核部署与升级

将生成的 `.deb` 包拷贝至目标开发板，执行以下命令完成安装与升级：

```
sudo apt install *.deb
```

5.2. 内核配置体系

EIC77 系列平台提供**标准化内核配置**，不同硬件平台对应独立的 `defconfig` 与设备树（DTS）文件。编译前需根据目标板型选择正确的配置文件，以确保硬件功能与外设接口正常适配。

5.2.1. 平台配置对照表

平台	默认配置文件 (defconfig)	设备树文件 (DTS)
HiFive Premier P550	kernel/arch/riscv/configs/eic7700_pm_defconfig	kernel/arch/riscv/boot/dts/eswin/eic7700-hifive-premier-p550.dts

平台	默认配置文件 (defconfig)	设备树文件 (DTS)
EBC77 SBC	kernel/arch/riscv/configs/eic7700_pm_defconfig	kernel/arch/riscv/boot/dts/eswin/eic7700-sbc-a1.dts
EIC7700FG EVB LP5	kernel/arch/riscv/configs/eic7700_pm_defconfig	kernel/arch/riscv/boot/dts/eswin/eic7700-evb-a2.dts
EBC7702 Mini-DTX	kernel/arch/riscv/configs/eic7702_pm_defconfig	kernel/arch/riscv/boot/dts/eswin/eic7702-d560.dts

5.2.2. 定制化内核配置

在产品开发中，默认配置往往需要根据业务需求进行裁剪、功能开启/关闭、驱动添加/移除、参数调整。通过定制化配置，可以减小内核体积、提升启动速度、降低功耗，并开启项目所需的专用驱动。

开发流程：

- 1. 修改配置：直接编辑平台对应的 defconfig 文件，或执行 make menuconfig 进行可视化配置。
- 2. 修改设备树：根据硬件变更调整 .dts 文件，描述新的硬件连接关系。
- 3. 重新编译：执行 make_kernel 生成包含最新配置的 Debian 安装包。
- 4. 部署验证：在目标板上安装新内核并重启验证。

内核更新标准流程：修改配置 → 编译构建 → DEB 打包 → 板端升级。

5.2.3. 高级定制与扩展

本节将从零开始，完整演示如何为开发板创建并配置专属的内核配置文件

添加板型选项：

在编译脚本 setenv.sh 中添加自定义开发板枚举选项：

```
# 添加自定义板型到列表
echo "board list:"
echo [1] eic7700-evb
echo [2] eic7700-hifive-premier-p550
echo [3] eic7702-d560
echo [12] eic7700-test
echo [0] Exit
```

添加板型定义：

在脚本选择逻辑中增加自定义板型匹配与参数配置：

```
# 板型选择判断
do
    elif [ "$CHOICE" == "12" ];then
        export board_name=eic7700-test
        export uboot_config=eic7700_test_config
        break
    fi
done
```

配置内核编译选项：

在内核编译函数中指定自定义板型对应的 defconfig：

```
function make_kernel()
{
```

```
if [[ "$board_name" =~ "eic7700-test" ]]; then
    linux_defconfig=eic7700_test_defconfig
fi
}
```

添加内核配置与设备树文件：

创建并放置对应的内核默认配置与设备树文件：

```
# 内核 defconfig 路径
kernel/arch/riscv/configs/eic7700_test_defconfig

# 设备树 DTS 路径
kernel/arch/riscv/boot/dts/eswin/eic7700-test.dts
```

5.3. 设备树 Overlay

设备树 Overlay 是在系统启动阶段，由固件或引导程序将多个设备树片段（.dtbo）与基础设备树（Base DTB）预先合并，再启动内核的配置方式。这种方式无需重新编译完整的内核 DTB，便于硬件功能的动态扩展（如接入不同的摄像头模组）。

5.3.1. 添加新的 Overlay DTS

1. 将 Overlay 设备树源文件（DTS）存放至内核目录：arch/riscv/boot/dts/eswin/overlays/
2. 修改该目录下的 Makefile，添加新增的 overlay 配置项，确保其能被编译系统识别。

5.3.2. 编译与部署

在内核编译与安装过程中，编译生成的 DTBO 文件会自动部署到系统目录：/boot/dtbs/linux-image-\$(kernel_version)/eswin/overlays/

5.3.3. 平台差异配置

EIC77 系列 SoC 在 **Debian** 与 **Ubuntu** 系统中使用不同引导流程，Overlay 配置方式有所区别：

Debian 系统（extlinux.conf 引导）

基于 extlinux.conf 实现标准 Overlay，无需额外脚本，U-Boot 原生支持。

1. 确保 U-Boot 已配置临时 DTB 合并内存空间：fdtoverlay_addr_r=0x100000000
2. 编辑 /boot/extlinux/extlinux.conf，添加：fdtoverlays /路径/xxx.dtbo /路径/yyy.dtbo
3. 重启生效
4. 恢复基础设备树：删除 fdtoverlays 配置项并重启

```
eswin@eswin:~$ cat /boot/extlinux/extlinux.conf
default l0
menu title U-Boot menu
prompt 2
timeout 50

label l0
    menu label RockOS GNU/Linux 6.6.115-eic7x-2025.12
    linux /vmlinuz-6.6.115-eic7x-2025.12
    initrd /initrd.img-6.6.115-eic7x-2025.12
    fdt dir /dtbs/linux-image-6.6.115-eic7x-2025.12/
```

```
fdtoverlays=/dtbs/linux-image-6.6.115-eic7x-2025.12/eswin/overlays/  
↪ eic7700-sbc-a1-camera-ov5647-overlay.dtbo
```

```
append root=UUID=80a5a8e9-c744-491a-93c1-4f4194fd690a console=tty0 console=ttyS0,115200  
↪ rootwait rw earlycon selinux=0 LANG=en_US.UTF-8 audit=0
```

Ubuntu 系统 (GRUB 引导)

GRUB 无法直接向 U-Boot 传递 Overlay 参数，因此采用**系统内预合并**方案。

1. 在 `/boot/` 目录下创建/编辑 `overlay.txt`
2. 配置规则：
 - `enable_overlay=1`：启用 Overlay 合并
 - `dtbo=文件路径`：指定要合并的 `.dtbo`（多个可空格分隔）
 - 任一 DTBO 合并失败，自动回退到基础设备树
 - `enable_overlay=0` 或不配置 `dtbo`：使用原始基础设备树
3. 执行合并命令：`merge_dtbo`。
4. 重启后生效。

5.4. 驱动开发指引

在内核开发过程中，除了核心架构与配置外，开发者通常需要针对具体的硬件外设进行驱动开发或适配，例如 GPIO、I2C、UART、Ethernet 等。

针对这些具体的外设驱动开发，包括但不限于驱动框架解析、设备树绑定 (Device Tree Bindings)、接口调试方法等深度内容，请参考专门的《驱动开发者手册》。该手册提供了比内核配置更深层次的开发指导，是进行 BSP 层和外设驱动开发的核心参考资料。

6. 系统开发指南

本章将深入 EIC77 系列 SoC 平台的系统级开发，涵盖内存管理、功耗调控等核心主题。内容旨在为底层系统工程师提供从硬件资源配置到系统稳定性保障的全景式指引。

6.1. 内存管理：MMZ 配置调整

在某些开发板（如部分搭载 **16GB 内存** 的板卡）中，系统默认划出 **6GB** 作为 **MMZ (Media Memory Zone)** 空间，这段内存对系统中 Top、Free 等工具是不可见的。在实际项目开发中，开发者常需根据业务需求（如视频缓存、AI 推理）调整 MMZ 的大小，具体可参考《ESSDK 内存用户手册》。

6.1.1. 核心约束与硬件信息

- 物理内存总量：16GB
- 内存地址范围：0x80000000 ~ 0x47FFFFFF
- 总内存结束地址：0x480000000
- 核心约束：MMZ 配置必须满足 起始地址 + 大小 <= 0x480000000

6.1.2. 查看当前内存布局

在 U-Boot 命令行中，可通过 `bdinfo` 和 `lmb_dump_all` 查看详细的内存分配情况：

```
=> bdinfo
boot_params = 0x0000000000000000
.....
lmb_dump_all:
memory.cnt = 0x1 / max = 0x10
memory[0] [0x80000000-0x47ffffff], 0x40000000 bytes flags: 0
reserved.cnt = 0x3 / max = 0x10
reserved[0] [0x80000000-0x8007ffff], 0x00080000 bytes flags: 4
reserved[1] [0xdfff0000-0xe1ffffff], 0x02010000 bytes flags: 4
reserved[2] [0xe9ce2000-0x47ffffff], 0x39631e000 bytes flags: 0
devicetree = board
```

确认 `memory[0]` 的地址范围为 0x80000000 至 0x47FFFFFF，总大小为 16GB。

6.1.3. 在 U-Boot 中调整 MMZ 大小

上电时快速按任意键进入 U-Boot 命令行，通过 `fdt` 命令配置 MMZ。

配置示例（以调整为 11GB 为例）：

```
# 设置环境变量，固化配置
setenv fdt_cmd "fdt mmz mmz_nid_0_part_0 0x1c0000000 0x2c0000000"
setenv bootcmd "run fdt_cmd;${bootcmd}"
saveenv
```

- 起始地址：0x1c0000000
- 大小：0x2c0000000 (11GB)
- 校验：0x1c0000000 + 0x2c0000000 <= 0x480000000

提示

若配置后系统启动异常，请检查 `addr + size` 是否大于 0x480000000。若参数设置错误导致异常，可恢复出厂环境变量：

```
env default -a
saveenv
```

6.1.4. 验证配置生效

系统启动后，执行以下命令验证 MMZ 大小是否已更新为 11G：

```
eswin@eswin:~$ cat /proc/eswin/vb
Module: [VB], Build Time[xx]
-----MMZ REGION CONFIG-----
memblock: mmz_nid_0_part_0, total size(0x2bffd000), free mem size(0x2bffd000), peak used size(0x0)
-----POOL CONFIG-----
```

6.2. 功耗调节与热管理

在高负载、高性能业务场景下，芯片功耗与发热量会显著上升。若散热条件受限，温度持续升高可能引发系统降频、重启甚至硬件损伤。为保障芯片长期稳定运行，需建立基于温度反馈的动态功耗管理机制。

6.2.1. 功耗调节整体方案

EIC77 系列 SoC 采用分层软件架构实现功耗管理：

- 1. **温度监测层**：通过 PVT 硬件模块实时获取芯片温度。
- 2. **内核驱动层**：基于 Linux 标准框架实现频率/电压控制（CPU 使用 cpufreq，外设使用 devfreq）。
- 3. **应用层工具**：提供 es_clk 脚本一键查询/设置各模块时钟。

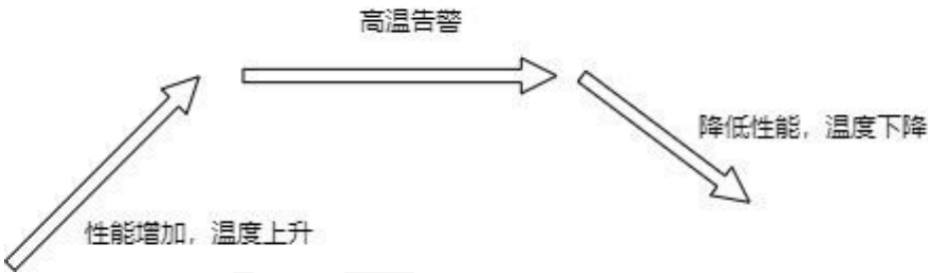


图 6: 业务场景下功耗调节示意

6.2.2. 芯片温度监测

芯片内置 **PVT (Process-Voltage-Temperature)** 模块，温度数据通过 hwmon sysfs 节点查看：

```
eswin@eswin:~$ cat /sys/class/hwmon/hwmon0/label
pvt0
eswin@eswin:~$ cat /sys/class/hwmon/hwmon0/temp1_input
33551
```

- **输出单位**：毫摄氏度 (m °C)
- **示例**：33551 → 33.551 °C

6.2.3. 驱动层功耗调节方法

6.2.3.1. Devfreq 模块调频（NPU/DSP/VENC/VDEC 等）

EIC77 系列 SoC 使用 `userspace` 策略，应用层可直接读写节点控制频率。

支持模块与节点路径:

模块	对应 devfreq 节点
DSP0	/sys/class/devfreq/52280400.dsp_subsys:es_dsp@0/
DSP1	/sys/class/devfreq/52280400.dsp_subsys:es_dsp@1/
DSP2	/sys/class/devfreq/52280400.dsp_subsys:es_dsp@2/
DSP3	/sys/class/devfreq/52280400.dsp_subsys:es_dsp@3/
VDEC	/sys/class/devfreq/soc:video-decoder0@50100000/
VENC	/sys/class/devfreq/soc:video-encoder@50110000/
DEWARP	/sys/class/devfreq/51020000.dewarp/
HAE	/sys/class/devfreq/50140000.g2d/
NPU	/sys/class/devfreq/51c00000.eswin-npu/

常用操作:

```
# 查看支持的频率
cat /sys/class/devfreq/xxx/available_frequencies
# 查看当前频率
cat /sys/class/devfreq/xxx/cur_freq
# 设置目标频率
echo userspace > /sys/class/devfreq/xxx/governor
echo 目标频率 > /sys/class/devfreq/xxx/userspace/set_freq
```

以 NPU 模块为例:

```
# 查看当前频率
cat /sys/class/devfreq/51c00000.eswin-npu/cur_freq
1500000000
# 查看支持的频率
cat /sys/class/devfreq/51c00000.eswin-npu/available_frequencies
520000000 750000000 1040000000 1188000000 1500000000
# 设置频率为 1.04GHz
echo userspace > /sys/class/devfreq/51c00000.eswin-npu/governor
echo 1040000000 > /sys/class/devfreq/51c00000.eswin-npu/userspace/set_freq
```

6.2.3.2. CPUFreq 调频（CPU 核心）

CPU 频率通过标准 `cpufreq` 框架管理，同一 DIE 内所有核心共享频率。

安装工具:

```
sudo apt install linux-cpupower
```

查看与设置 CPU 频率:

```
# 查看 CPU 频率信息
cpupower -c all frequency-info
# 设置所有 CPU 频率为 800MHz
cpupower --cpu all frequency-set --freq 800MHz
```

6.2.4. 便捷工具：es_clk 脚本

平台提供 /usr/bin/es_clk 工具，无需记忆复杂节点路径，一键查询/设置所有可调频模块。

命令	功能
es_clk -g	查询所有模块当前时钟
es_clk -g <模块名>	查询单个模块（如 NPU）
es_clk -q <模块名>	查询模块支持的频点列表
es_clk -s <模块名> <频率>	设置模块频率

支持模块名（大写）： VENC、VDEC、DSP0、DSP1、DSP2、DSP3、DEWARP、HAE、NPU

示例：

```
# 查询 NPU 当前频率和支持的频点
es_clk -g NPU
es_clk -q NPU
# 设置 NPU 频率为 1.04GHz
es_clk -s NPU 1040000000
```

6.2.5. 硬件监控与风扇控制

6.2.5.1. hw-watcher 硬件状态监控

es_hw_watcher 是面向 EIC77 系列 SoC 的硬件状态实时监控工具，可动态采集、可视化展示设备关键运行信息。

安装与运行：

```
sudo apt install es-hw-watcher
sudo /opt/eswin/bin/es_hw_watcher
```

输出内容包括芯片温度、工作电压、模块利用率、数据带宽等，便于业务调试与性能定位。

6.2.5.2. 风扇转速调节

EIC77 系列 SoC 提供 sysfs 标准节点实现风扇控制，支持查询风扇控制节点、手动设置转速占空比。

定位风扇控制节点：

通过 label 节点确认 fan_control 对应的 hwmon 路径：

```
cat /sys/class/hwmon/hwmon*/label

# 典型输出
pvt0
```

```
pvt1  
d0_board_tmp  
npu_vdd  
fan_control
```

设置风扇转速：

风扇转速通过 **pwm1** 节点配置，取值范围 **0 – 99**（数值越大转速越快）。

```
# 设置为 50% 转速  
echo 50 > /sys/class/hwmon/hwmon4/pwm1  
# 设置为 99% 转速  
echo 99 > /sys/class/hwmon/hwmon4/pwm1
```

6.2.6. 高温保护流程建议

建议结合温度监测与调控手段，制定如下保护策略：

1. 周期性读取温度： `cat /sys/class/hwmon/hwmon0/temp1_input`
2. $\geq 75^{\circ}\text{C}$ ：提高风扇 PWM、降低 CPU 频率、降低 NPU 频率。
3. $\leq 65^{\circ}\text{C}$ ：逐步恢复 CPU/NPU 默认频率。